Meta-Probing Utility Predictor (MPUP): Predicting the Utility of Retrieved Passages for RAG without Fine-Tuning the LLM

1st Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

2nd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

3rd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

4th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

5th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

6th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

Abstract—Retrieval-Augmented Generation (RAG) systems rank passages by topical relevance, yet relevance and usefulness are not the same thing. Even a passage that matches a query may fail to help the model answer. Recent utility-aware RAG methods proven to be impractical because existing methods either fine-tune a reranker or retriever's embedding model for every target LLM, or run the target LLM on every candidate passage at selection time, which takes up high cost, memory, and computation. Meta-Probing Utility Predictor (MPUP) is a method to predict LLM-specific retrieval utility without fine-tuning the target model.

MPUP turns each query–passage pair into 14 inexpensive features: retriever scores, passage statistics, entity overlap, an NLI check against a hypothetical answer, and the change in the next-token entropy of one LLM caused by the passage. Passages are ranked by a small regressor trained on within-query deviations, and the target LLM is never fine-tuned. On eleven question-answering datasets labelled with the true utility, with the predictor selected by nested leave-one-dataset-out, MPUP reaches a within-query Spearman correlation of 0.33 with the true utility, against 0.12 for BM25 and 0.29 for dense similarity. Over dense similarity, the gain is small (+0.04, 95% CI $[+0.02, +0.06]$), and the predictor stated first in this paper is worse than dense similarity alone. With the configuration frozen, a second sample of questions gives 0.31 against 0.28 for dense similarity. Peak GPU memory is 21–22 GiB in bfloat16; with 4-bit NF4 weights it peaked at about 7 GiB on a small sample.

Index Terms—retrieval-augmented generation, passage utility, utility prediction, passage selection, logit probing

I. Introduction

Large language models (LLMs) are Transformer-based neural networks pre-trained on massive unlabeled text so that they are adapted to specific tasks through prompting rather than task-specific retraining [1], making one model can do reasoning, summarization, and generation tasks that previously require separate architectures [2]. LLMs are used in multiple areas, one of them is LLM for personalized tutoring and automated feedback in education [3]. LLMs can also be used in reshaping professional practice, research workflows, and policy [4].

LLMs often produce hallucinations which are outputs unfaithful to the source or factually incorrect [5], because their knowledge is stored and frozen at training time, making them cannot be corrected without retraining [6]. Retrieval-Augmented Generation (RAG) addresses this problem by retrieving relevant documents from external sources and do generation on that retrieved evidence, so the model draws on up-to-date information instead of only memorized parameters [7]. RAGs have multiple methods to retrieve the documents, one of them is traditional RAG which select documents purely by topical relevance where the contents match the queries, which works worse than RAG systems where the documents are selected by their utility for the model [8].

Selecting documents or passages based on their utility in RAG is done by seeing whether the passage providing it as evidence measurably improves a LLM's ability to generate an accurate answer compared to answering without any evidence [8]. To measure this, two cases need to be compared. In the first, the model answers from its internal memory alone. In the second, the same model answers with a specific retrieved passage available to it. Both answers are scored by a standard correctness metric, and a passage counts as useful only when the second answer is more accurate than the first. Applying the same test to a large pool of candidates leaves only the passages that actually help that model [8].

However, how can a practitioner estimate passage utility under realistic, low-budget conditions? Existing methods are too expensive for practical deployment, because they

require a 32-billion-parameter teacher model to produce supervision [9], completely retraining the retriever's embedding model using complex objectives [10], or running the target LLM twice for every single query [11].

There are cheap alternatives method compared to LLM sampling. Semantic Perplexity (SePer) measures how useful a retrieved passage by observing how much its semantic perplexity changes after the passage is provided, but it still requires running an evaluation model, which adds computation and latency, making it less suitable for real-time passage selection [12]. Another approach proposes a lightweight neural model that predicts passage utility for a target Question Answering (QA) model, but its predictor is trained for a specific QA model, making it difficult to apply to other QA models [13]. Another method is a feature-based predictor combining Query Performance Prediction (QPP), perplexity, readability, and document quality features using linear regression to predict retrieval utility, but this method achieves relatively low accuracy for utility prediction, with Spearman $\rho \approx 0.20$–0.24 for retrieval utility on Natural Questions [14]. Low accuracy and unapplicable solutions show that none of these methods has solved the problem yet.

Several other methods have been explored. There are LURE-RAG which introduces a lightweight LambdaMART reranker trained with a listwise utility loss to optimize context ordering, but relying on complex listwise utility supervision requires generating query and passage combinations, making the labeling cost expensive [15]. Another approach is RRPO, where passages selection treated as a sequential decision process trained via reinforcement learning directly from LLM feedback, which also which takes up big part of the memory and computation [16]. Existing methods either require substantial memory and computational resources, or incur high costs.

Meta-Probing Utility Predictor (MPUP), proposed in this paper, closes the gap. MPUP is a method to predict LLM-specific retrieval utility which means it can select the most useful passages for models without fine-tuning any model. Instead of having the target LLM answer with and without every passage [11] or scoring each passage with a separate evaluation model [12], MPUP evaluates a passage's usefulness by looking at 14 practical clues, such as document statistics, meaning overlap, and agreement with a hypothetical answer. Only three of them read the logits of an LLM, through one forward pass per passage (logit probing). MPUP runs its two LLM-based steps, one short generation per query and the probing pass, with one shared model on a single GPU and does not train any LLM. Section IV reports the measured memory footprint and extraction time.

Contributions of this paper are threefold. The first is that many cheap features, used together, rank passages by true utility better than the retriever's own score (BM25) and slightly better than a strong dense-similarity baseline, without a utility-specific model and without having an LLM generate an answer for each passage. Compared with dense similarity, the gain is small, and we report it with confidence intervals. Execution on a single GPU is also possible, and the measured peak memory is reported in Section IV. The second is a predictor that is trained once and is not retrained when the LLM that answers the question changes. Training and evaluation use one source LLM at a time, so transfer to other target LLMs is not tested in this paper. The third is that the predictor can also be trained on annotations that already exist in the dataset, which are cheaper than labels produced by running the target LLM over every passage [17]. Trained this way, the predictor beats BM25 on nine of eleven datasets but not dense similarity.

Remaining sections are organized as follows. Section II reviews related work on retrieval-augmented generation, utility-based passage selection, and lightweight utility prediction. Section III describes the proposed Meta-Probing Utility Predictor, covering the 14 features, the predictor architecture, and the model and hardware setup. Section IV presents the experimental setup, datasets, and evaluation metrics, followed by the results and analysis, while Section V concludes the paper and outlines directions for future work.

II. Literature Review

LLM can only use the knowledge it learned during training, and this knowledge cannot be updated without retraining. RAG solves this by first retrieving documents from an external source, then ask the LLM to write its answer based on those documents [7]. In traditional RAG, passages are chosen by relevance, which means how closely their content matches the query.

However, a relevant passage is not always a helpful one. Recent work shows that choosing passages by their utility for the model gives better results than choosing them by relevance alone [8]. Utility is measured by comparing two answers from the same LLM. The first answer is written from the model's own memory. The second is written with the candidate passage. Correctness metric scores both answers, and the passage is useful only if the second answer is more accurate [8].

Utility also depends on what the model already knows. Passage may help one LLM but add nothing for another LLM that already knows the answer. So, utility belongs to a pair of passage and model, not to the passage alone. To estimate it in practice, we can either test the target LLM directly or approximate it with cheaper signals.

Methods that estimate passage utility can be grouped into four types. Utility estimation by large models or utility-trained retrievers is the first type. Zhang et al. [9] distill the utility judgments of a 32-billion-parameter teacher LLM into smaller student models, and Sandhu et al. [10] retrain the retriever's embedding model with complex objectives so that its ranking follows utility. Both

reach utility-aware selection, but at the cost of heavy training, and the former also needs a large teacher model to produce its supervision, which is hard to afford under low-budget conditions. Learned rerankers and selection policies trained from utility feedback form the second. LURE-RAG introduces a lightweight LambdaMART reranker trained with a listwise utility loss [15], while RRPO treats passage selection as a sequential decision process optimized by reinforcement learning from LLM feedback [16]. LURE-RAG depends on expensive query-passage labeling, and RRPO consumes substantial memory and computation.

Utility estimation by directly involving the target LLM is the third. Zhang et al. [11] run the target LLM twice for every query to compare its behavior with and without the passage, and SePer measures how much the semantic perplexity changes once the passage is provided [12]. Methods of this type are closest to the definition of utility as improvement over the model's internal memory, but they add inference cost and latency at selection time, so they are less suitable for real-time use. Lightweight utility predictors form the fourth. Perez-Beltrachini and Lapata [13] propose a lightweight neural model that predicts utility for a target QA model, yet it is trained for one specific model and is difficult to transfer to others. Tian et al. [14] combine Query Performance Prediction (QPP), perplexity, readability, and document quality features in a linear regression, which is cheap but reaches only Spearman $\rho \approx 0.20$–0.24 for retrieval utility on Natural Questions.

MPUP is compared to eight previous studies which were based on six criteria, presented in Table I. C1 is a lightweight model that runs on limited hardware. C2 is no fine-tuning of the LLM or the retriever. C3 is no additional LLM run for every query at selection time. C4 is utility estimation accuracy that is competitive with methods that probe the target LLM directly. C5 is a single predictor that works for several target LLMs without retraining. C6 is training on annotations that already exist in the dataset instead of labels generated by the target LLM.

TABLE I
Positioning of MPUP to Previous Studies

Method	C1	C2	C3	C4	C5	C6
Zhang et al. [9]	×	×	✓	×	×	✓
Sandhu et al. [10]	×	×	✓	×	×	✓
Zhang et al. [11]	×	✓	×	✓	×	✓
Dai et al. [12] (SePer)	×	✓	×	✓	×	✓
Perez-Beltrachini & Lapata [13]	✓	✓	✓	×	×	✓
Tian et al. [14]	✓	✓	✓	×	×	×
LURE-RAG [15]	✓	✓	✓	×	✓	×
RRPO [16]	×	×	✓	×	×	✓
MPUP (ours)	✓	✓	✓	✓	✓	✓

Table I shows how the studies differ. Methods that reach high accuracy through the target LLM [11], [12] fail the lightweight and per-query cost criteria, while lightweight methods [13], [14] either remain tied to one model or lose accuracy. LURE-RAG [15] is the one study that reports reusing a predictor, trained on the utilities of one LLM, for other LLMs (its fourth research question), but its labels come from running an LLM. MPUP is designed to meet all six criteria; Section IV reports the measurements behind each of them and their limits.

Utility-based selection is only useful if people can actually run it. Many practitioners have access to only a single GPU and a limited budget, not a training cluster. Methods that need very large models, retriever retraining, or two LLM runs for every query are out of reach for them. Methods that are cheap but tied to one LLM create another problem, because the predictor must be rebuilt whenever the LLM is replaced. Lightweight methods with low accuracy are also not helpful, since they often pick the wrong passages. Every barrier above would be removed by a predictor that is lightweight and does not have to be rebuilt for each new LLM, provided that its ranking is good enough to help. MPUP is designed for this purpose by combining many cheap features into one predictor, and Section IV measures how good its ranking is.

III. Methodology

Fig. 1 shows the MPUP pipeline. For each query, MPUP first asks an LLM M to write one short hypothetical answer. Then, for every candidate passage, it extracts 14 features from five feature families. Next, the trained regressor maps these features to a utility score, and an optional calibration step adjusts the scale of the score. Passages are ranked by the score, and the top passages are placed in the prompt of the target LLM. No fine-tuning of the target LLM is ever needed, and the question does not need to be answered twice for every passage.

Passage utility measures whether a passage, given as evidence, improves the accuracy of an LLM compared with answering without any evidence [8]. Let q be query, d candidate passage, and L target LLM. Utility of d for L on query q is the difference in generation accuracy [8]:

$$u(d; L, q) = \mathrm{Acc}(L(q, d)) - \mathrm{Acc}(L(q, \varnothing)). \tag{1}$$

Where $L(q, \varnothing)$ is the answer of L from its own memory alone, and $L(q, d)$ is the answer of L when it reads d. $\mathrm{Acc}(\cdot)$ is a standard correctness metric. Passage is useful only if reading it improves the model, which means $u > 0$. Zero utility means the passage adds nothing, and a negative value means it makes the answer worse.

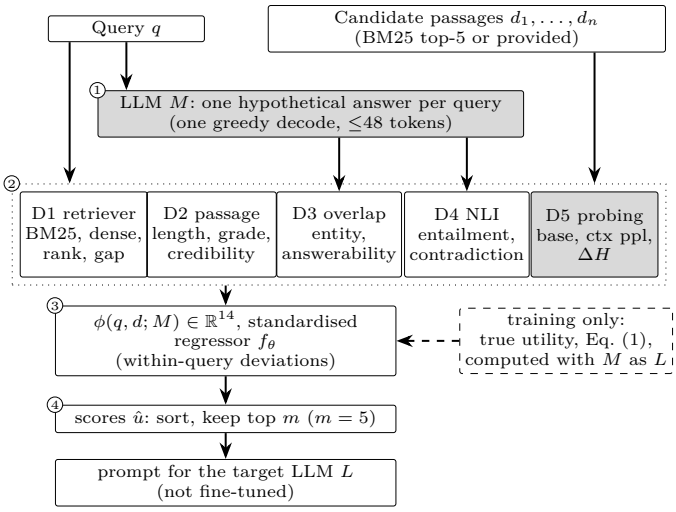


Fig. 1. Overview of MPUP. (1) M writes one hypothetical answer per query. (2) Five feature groups are computed for every passage; D3 and D4 use the answer, D5 probes M. (3) The regressor maps the 14 features to a utility score. (4) The top m passages go into the prompt of the target LLM (not fine-tuned). Shaded: uses M, one shared model. Dashed: training only.

If we test a pool of candidate passages $\mathcal{C}$ in this way and keep only the passages that help L, we obtain the gold utilitarian set of L [8]:

$$\widetilde{\mathcal{G}}_q^{(L)} = \{d \in \mathcal{C} \mid u(d; L, q) > 0\}. \tag{2}$$

Acc is the average of relaxed exact match and token-level F1 against the gold answers:

$$\mathrm{Acc}(a) = 0.5 \cdot \mathrm{EM}_r(a) + 0.5 \cdot \mathrm{F1}(a). \tag{3}$$

Relaxed exact match counts an answer as correct when a gold answer appears inside it as a token span. Relaxed matching is used because instruction-tuned LLMs answer in full sentences, and strict exact match would mark most of them wrong, which pushes almost every utility to zero. Values of u lie in $[-1, 1]$.

Building $\widetilde{\mathcal{G}}$ directly needs two LLM runs for every query-passage pair, which is too expensive for real-time selection. MPUP predicts the utility instead, so that passages in $\widetilde{\mathcal{G}}$ are ranked first:

$$\hat{u}(d; q) = f_\theta(\phi(q, d; M)), \tag{4}$$

where $\phi(q, d; M) \in \mathbb{R}^{14}$ is the feature vector and f_θ is the predictor. M is the LLM used for the two LLM-based steps, the hypothetical answer and logit probing. Either the target LLM L or a fixed model can serve as M. In our experiments, M is Qwen2.5-7B-Instruct. Predictor f_θ is trained once and is not retrained when L changes.

Table II lists the 14 features. D1 describes how the retriever scored the passage: its BM25 score (divided by the highest score of the query), its dense similarity, its rank, and its score gap to the next passage. D2 describes the passage itself: token length, Flesch–Kincaid grade, and a source credibility score. D3 measures meaning overlap. Entity overlap is the fraction of query entities, found by spaCy, that also appear in the passage. Answerability is the fraction of content words of the hypothetical answer that appear in the passage. D4 checks faithfulness with a DeBERTa-v3-small NLI model. Here, the passage is the premise and the hypothetical answer is the hypothesis, and we keep the entailment and contradiction probabilities. D5 probes the LLM M. Next-token entropy is computed at the end of two prompts: the query alone (H_{base}) and the passage followed by the query (H_{ctx}). D5 features are the two perplexities $\exp(H)$ and the difference $\Delta H = H_{\text{base}} - H_{\text{ctx}}$. Generated once per query, the hypothetical answer is shared by D3 and D4.

TABLE II
Features of MPUP

Group	Features
D1 Retriever	BM25 score, dense similarity, rank, score gap
D2 Document quality	Token length, Flesch–Kincaid grade, source credibility
D3 Semantic overlap	Entity overlap, HyDE answerability
D4 Faithfulness	Entailment probability, contradiction probability
D5 LLM probing	Base perplexity, context perplexity, ΔH

Standardized features are passed to a regressor. Two kinds are considered. One kind, the reference configuration, is a gradient-boosted regression tree model (200 trees, depth 5, learning rate 0.05, subsample 0.8) trained on the raw features. Another kind trains on within-query deviations: the mean of the query is subtracted from every feature and from the label before fitting, and at inference the candidates of the query being ranked form that group. Much of the raw utility is an offset per query (a question that the LLM already answers makes every passage useless), which no reranker can use. Centred models spend no capacity on it, and their scores mean "better or worse than the average candidate". The configuration is chosen from a grid of twelve (ridge regression with three regularization strengths and gradient-boosted trees of three sizes, each with and without centring) by nested leave-one-dataset-out, described in Section IV.

Two kinds of label are used. In the main experiments the predictor is trained on the true utility (1) of the training datasets. When no LLM labels are wanted, it can instead be trained on labels derived from existing annotations. RAGBench marks which sentences of each document were used to produce the answer. For passage d, the label is the fraction of its sentences that are marked:

$$\tilde{u}(d) = \frac{|S_d \cap S_{\text{used}}|}{|S_d|}, \tag{5}$$

where S_d is the set of sentences of d and S_{used} is the set of marked sentences. No LLM is run to produce these labels. True utility (1) is computed with the same LLM

M acting as the source LLM. Generated once per query, the no-context answer is reused for all passages of that query.

Optionally, $\hat{u}$ can be calibrated when labels from the target LLM are available for k passages. Coefficients α and β are fitted by linear regression on these k examples, and the calibrated score is

$$\hat{u}' = \alpha \cdot \hat{u} + \beta. \tag{6}$$

Calibration changes the scale of $\hat{u}$, which matters for the threshold τ. Rankings do not change as long as $\alpha > 0$, because the order of the scores stays the same. Calibration is not used in the main results.

Both LLM-based steps, hypothetical answer generation and logit probing, share one model M: Qwen2.5-7B-Instruct, run in bfloat16 on a single NVIDIA RTX 6000 Ada GPU (48 GB). Sharing one copy avoids loading a second 7B model, and full-precision weights mean that no quantization error enters the probing and generation steps. Alternatively, the implementation can load M in 4-bit NF4 with double quantization to reduce memory; we report peak memory for both settings in Section IV. Also available is an Apple Silicon backend that uses a 4-bit Llama-3.2-3B-Instruct through MLX. Both the NLI model and the regressor are small and run without quantization.

At inference time, MPUP makes one short greedy decode per query (at most 48 tokens) for the hypothetical answer. For each passage it makes one probing pass and one NLI pass, then applies the regressor. Passages are sorted by score, and the top m ($m = 5$ by default) are written as numbered context before the query. Dropping passages with a score $\leq \tau$ is also supported. With the selected predictor, which is trained on within-query deviations, the score is relative to the other candidates of the same query, so $\tau = 0$ would drop roughly the less useful half of the candidates rather than the passages with negative utility. No threshold is therefore applied. Neither choice is evaluated by answer accuracy in this paper.

IV. Experimental Setup

A. Datasets

Table III lists the twelve datasets used in the experiments. Eleven of them have gold answers, so the utility of every query-passage pair is computed with Eq. (1). RAGBench has no gold answer and carries the sentence-annotation label of Eq. (5), so it is never pooled with the other eleven. For each dataset with gold answers, 100 questions are sampled with a fixed seed after removing yes/no questions (HotpotQA, 2WikiMultiHopQA) and questions whose text occurs more than once with different answers (PopQA). When a dataset provides no passages, the candidates are the top five passages by BM25 over the English Wikipedia corpus of 21,015,324 passages of 100 words [18]. HotpotQA, 2WikiMultiHopQA, and MuSiQue use all the paragraphs they provide.

B. Evaluation protocol

All metrics are computed within each query and then averaged over queries, because a reranker can only order the passages of one query. Queries are used when they have at least two passages and their true utilities are not all equal (747 of the 1,100 queries with gold answers). Spearman correlation ρ between the predicted score and the true utility serves as the main metric. Intervals are 95% percentile bootstrap intervals over queries (2,000 resamples), and differences between methods are paired on the same queries. Splits are always by query, never by row.

Because the predictor is chosen from data, its score is estimated with nested leave-one-dataset-out. One dataset is held out. Among the other ten, each of the twelve configurations is scored by leave-one-dataset-out and the best is kept. Refitted on those ten datasets, the best configuration is scored on the held-out one, whose labels are not used for the choice. Choosing the best configuration on the held-out dataset itself would have reported $\rho = 0.357$, which is 0.024 above the nested estimate. Since the grid was designed after an earlier run on the same questions with a smaller 3B model had shown that the reference configuration was weak, even the nested estimate is slightly optimistic. Results without any selection are also reported for the reference configuration.

C. Setup

Qwen2.5-7B-Instruct, run in bfloat16 on one NVIDIA RTX 6000 Ada GPU, serves as LLM M. Model M writes the hypothetical answer, produces the probing features, and acts as the source LLM that defines the true utility (greedy decoding, at most 32 new tokens, relaxed exact match). Multilingual-E5-large provides the dense feature and the dense baseline. Baselines are BM25 and dense cosine similarity used alone as ranking scores.

D. Results

Table IV shows the held-out results. MPUP, selected by nested leave-one-dataset-out, reaches a macro-average ρ of 0.333, against 0.289 for dense similarity, 0.117 for BM25, and 0.227 for the reference configuration. Over all 747 held-out queries, the paired gain over BM25 is +0.20 [+0.16, +0.24] and over dense similarity +0.04 [+0.02, +0.06]. Therefore, the gain over dense similarity is real but small, and on most single datasets its interval includes zero (Fig. 2). Dense similarity alone beats the reference configuration (raw features, 200 trees of depth 5): in pooled cross-validation the paired difference is -0.06 [-0.09, -0.04]. Two changes helped: training on within-query deviations and using lower capacity. For all eleven held-out datasets, the selection chose a centred model (a shallow tree ensemble six times, ridge regression five times).

a) Feature groups: Table V shows the change in ρ when one group is removed and the model is retrained, paired over queries. Retriever features (D1) matter most.

TABLE III
Datasets used in MPUP

Dataset	Source and split (size)	Candidate passages per question	Utility label	Questions / pairs used
RAGBench, HotpotQA subset [19]	galileo-ai/ragbench, config hotpotqa, test (390)	Provided: four documents	Sentence annotation (Eq. (5))	390 / 1,557
Natural Questions, open [20]	FlashRAG nq, test (3,610)	Top five by BM25	Eq. (1)	100 / 500
TriviaQA, unfiltered [21]	FlashRAG triviaqa, test (11,313)	Top five by BM25	Eq. (1)	100 / 500
WebQuestions [22]	FlashRAG web_questions, test (2,032)	Top five by BM25	Eq. (1)	100 / 500
EntityQuestions [23]	Authors' release, test (22,075)	Top five by BM25	Eq. (1)	100 / 500
PopQA [24]	FlashRAG popqa, test (14,267; 12,187 after duplicates)	Top five by BM25	Eq. (1)	100 / 500
SQuAD [25]	FlashRAG squad, dev (10,570)	Top five by BM25	Eq. (1)	100 / 500
AmbigQA [26]	FlashRAG ambig_qa, dev (2,002)	Top five by BM25	Eq. (1)	100 / 500
Bamboogle [27]	FlashRAG bamboogle, test (125)	Top five by BM25	Eq. (1)	100 / 500
HotpotQA, distractor [28]	hotpotqa/hotpot_qa, validation (7,405; 6,947 without yes/no)	Provided: ten paragraphs	Eq. (1)	100 / 1,000
2WikiMultiHopQA [29]	FlashRAG 2wikimultihopqa, dev (12,576; 11,268 without yes/no)	Provided: ten paragraphs	Eq. (1)	100 / 1,000
MuSiQue [30]	dgslibisey/MuSiQue, validation (2,417)	Provided: up to twenty paragraphs	Eq. (1)	100 / 1,997

TABLE IV
Within-query Spearman ρ with the true utility on held-out datasets (nested leave-one-dataset-out, Qwen2.5-7B-Instruct, bfloat16). Last column: paired difference to dense similarity with its 95% bootstrap interval. †Computed over all held-out queries, not as an average of the rows.

Held-out dataset	Queries	MPUP	Reference config.	Dense	BM25	MPUP − dense
2WikiMultiHopQA	94	0.24	0.16	0.21	0.08	+0.04 [-0.00, +0.08]
AmbigQA	61	0.43	0.32	0.42	0.08	+0.01 [-0.08, +0.10]
Bamboogle	48	0.39	0.20	0.28	0.10	+0.11 [-0.00, +0.23]
EntityQuestions	65	0.32	0.24	0.27	0.29	+0.05 [-0.03, +0.15]
HotpotQA	90	0.27	0.14	0.28	0.07	-0.00 [-0.05, +0.05]
MuSiQue	84	0.22	0.17	0.17	0.04	+0.05 [+0.01, +0.09]
NQ	56	0.44	0.26	0.35	0.03	+0.09 [-0.00, +0.20]
PopQA	44	0.56	0.37	0.51	0.13	+0.05 [-0.04, +0.13]
SQuAD	67	0.37	0.23	0.23	0.25	+0.14 [+0.05, +0.24]
TriviaQA	71	0.28	0.33	0.33	0.17	-0.05 [-0.13, +0.02]
WebQuestions	67	0.14	0.07	0.13	0.05	+0.01 [-0.11, +0.13]
Macro mean	747	0.333	0.227	0.289	0.117	+0.04 [+0.02, +0.06]†

With the selected configuration, the LLM probing group (D5) adds +0.02 [+0.00, +0.04] and the hypothetical-answer overlap (D3) adds +0.01 [+0.00, +0.03]. Document quality (D2) and NLI (D4) add nothing measurable. Most of the signal therefore comes from the retriever features, and each LLM-based group adds at most a few hundredths of ρ. Because the configuration was selected on the same datasets, the table is descriptive. Source credibility is constant (0.5) in all datasets, so effectively 13 features carry information.

TABLE V
Change in ρ when a feature group is removed (paired; positive means the group helps). Pooled 5-fold, 5-repeat cross-validation over queries.

Group	Selected config.	Reference config.
D1	+0.06 [+0.03, +0.09]	+0.10 [+0.07, +0.13]
D2	+0.00 [-0.00, +0.00]	-0.00 [-0.01, +0.01]
D3	+0.01 [+0.00, +0.03]	-0.01 [-0.02, -0.00]
D4	-0.00 [-0.00, +0.00]	-0.01 [-0.02, +0.01]
D5	+0.02 [+0.00, +0.04]	+0.02 [+0.00, +0.04]

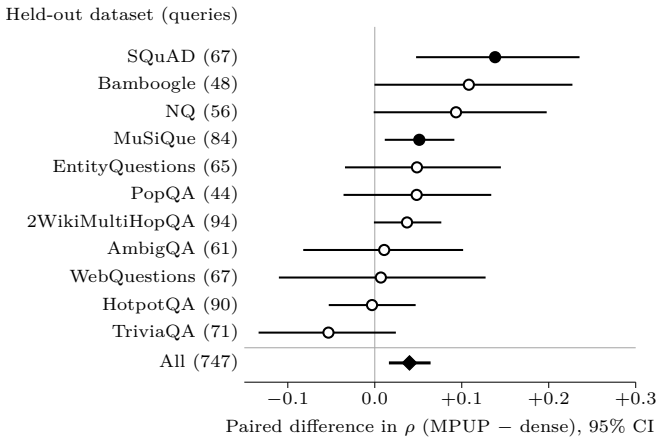


Fig. 2. Paired difference in within-query Spearman ρ between MPUP and dense similarity on each held-out dataset (nested leave-one-dataset-out, Qwen2.5-7B-Instruct), with 95% bootstrap intervals over queries. Filled markers: the interval excludes zero (MuSiQue, SQuAD); open markers: it includes zero. Last row (diamond): the difference over all 747 held-out queries together.

b) Training without LLM labels: Trained only on the RAGBench annotation label, with no label from the target datasets, the reference-configuration predictor reaches a macro-average ρ of 0.239. Compared with BM25 (0.117), the predictor is better on nine of eleven datasets, and it remains below dense similarity (0.289). Annotation labels are cheap, but on their own they do not give a better ranking than a dense retriever.

c) Another LLM M: Earlier, the same pipeline was run with Llama-3.2-3B-Instruct in 4-bit precision through MLX on Apple Silicon (same questions and candidates, 719 held-out queries). Macro-average ρ was 0.286 for MPUP, 0.256 for dense similarity, 0.110 for BM25, and 0.211 for the reference configuration. Paired against dense similarity, the gain was +0.02 [+0.00, +0.05], and the annotation-trained predictor reached 0.194. Methods keep the same ordering as with Qwen2.5-7B-Instruct, and the absolute values are lower. Because the two runs differ in the model, the quantization, and the hardware at the same time, the difference between them cannot be attributed to one of these.

d) Fresh questions with a frozen configuration: Designed after looking at the first sample of questions, the configuration grid makes the nested score above slightly optimistic. As a cleaner check, we drew a second sample of 100 questions per dataset with another seed (390 for RAGBench) and removed the questions already in the first sample (78 of the 100 Bamboogle questions, 0 to 8 in the other datasets). Fixed beforehand, the configuration of shallow gradient-boosted trees on within-query deviations (100 trees, depth 2) was fitted on the first sample of the eleven datasets with true utility and scored on the 693 rankable fresh questions without any selection. Macro-average ρ is 0.31, against 0.28 for dense similarity, 0.15 for BM25, and 0.19 for the reference configuration. Paired gains are +0.05 [+0.03, +0.08] over dense similarity, +0.16 [+0.12, +0.20] over BM25, and +0.11 [+0.08, +0.15] over the reference configuration. Ordering and size of the gain are the same as in the nested estimate. Because the check uses the same datasets, transfer to new datasets is not tested, and Bamboogle contributes only 6 fresh questions.

E. Cost

Table VI gives the peak GPU memory of the full pipeline (generation, labelling, probing, and NLI), measured on the datasets with the longest contexts (40 questions from MuSiQue and HotpotQA, batch size 32). Labelling and feature extraction for all twelve datasets (9,554 query–passage pairs, 7,997 of them labelled with the true utility) took 27 minutes in bfloat16 on one GPU, about 0.17 s per pair. Evaluation itself runs on the CPU. Over the whole extraction run for the second sample of questions (all twelve datasets), the peak in bfloat16 was 22.3 GiB. Neither the 4-bit setting end to end nor the behaviour on a card with 8 GB of memory was measured.

TABLE VI
Peak GPU memory of the pipeline with Qwen2.5-7B-Instruct.

Weights	Peak memory (GiB)
bfloat16	20.8
4-bit NF4	7.3

F. Limitations

Results use one source LLM per run and 100 questions per dataset (390 for RAGBench). One sample of questions underlies the main estimate, and the second sample checks it only for a frozen configuration on the same datasets, so the intervals reflect the variation among questions, not among models. Because true utility is defined by the same LLM M that produces the D5 features, whether a predictor trained with one source LLM ranks passages well for a different target LLM has not been tested. Answer accuracy when the top-m passages of MPUP are placed in the prompt was also not measured; only the quality of the ranking is evaluated. Extraction time above includes the labelling with the true utility; the latency of selection alone (one generation per query, then probing, NLI, and the regressor per passage) was not measured. The probing prompt (passage, then query) is truncated at 512 tokens from the right, so for 35 of the 9,554 pairs (0.37%) the query is cut off and their D5 features are computed without it. Candidate sets are small (4 to 20 passages), and in the datasets without provided passages they are the top five of BM25, which limits how much any reranker can change.

V. Conclusion

MPUP is a predictor of the utility of a retrieved passage for an LLM that combines 14 inexpensive features and does not fine-tune the target model. Across eleven datasets

labelled with the true utility, a shallow predictor trained on within-query deviations ranks passages better than BM25 and slightly better than dense similarity, while the predictor first specified is worse than dense similarity alone. Most of the signal comes from the retriever features, and the LLM-based features add a small amount. Fitting on one GPU, the pipeline peaks at 21–22 GiB in bfloat16 and about 7 GiB in 4-bit precision on a small sample. Future work should test transfer between source and target LLMs, measure answer accuracy with the selected passages, and evaluate on datasets that are not in the pool.

References

[1] B. Min, H. Ross, E. Sulem, A. P. B. Veyseh, T. H. Nguyen, O. Sainz, E. Agirre, I. Heintz, and D. Roth, "Recent advances in natural language processing via large pre-trained language models: A survey," ACM Comput. Surv., vol. 56, no. 2, pp. 1–40, Sep. 2023.

[2] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang et al., "A survey on evaluation of large language models," ACM Trans. Intell. Syst. Technol., vol. 15, no. 3, pp. 1–45, Mar. 2024.

[3] E. Kasneci, K. Sessler, S. Küchemann, M. Bannert, D. Dementieva et al., "ChatGPT for good? On opportunities and challenges of large language models for education," Learn. Individ. Differ., vol. 103, p. 102274, Apr. 2023.

[4] Y. K. Dwivedi, N. Kshetri, L. Hughes, E. L. Slade, A. Jeyaraj et al., "Opinion paper: "So what if ChatGPT wrote it?" Multidisciplinary perspectives on opportunities, challenges and implications of generative conversational AI for research, practice and policy," Int. J. Inf. Manage., vol. 71, p. 102642, Aug. 2023.

[5] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," ACM Comput. Surv., vol. 55, no. 12, pp. 1–38, 2023.

[6] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng et al., "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," ACM Trans. Inf. Syst., vol. 43, no. 2, pp. 1–55, 2025.

[7] W. Fan, Y. Ding, L. Ning, S. Wang, H. Li, D. Yin, T.-S. Chua, and Q. Li, "A survey on RAG meeting LLMs: Towards retrieval-augmented large language models," in Proc. 30th ACM SIGKDD Conf. Knowl. Discovery Data Mining (KDD), 2024, pp. 6491–6501.

[8] H. Zhang, M. Tang, K. Bi, and J. Guo, "Beyond relevance: Utility-centric retrieval in the LLM era," in Proc. 49th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval (SIGIR), Tutorial, 2026.

[9] H. Zhang, K. Bi, J. Guo, J. Zhang, S. Wang, D. Yin, and X. Cheng, "Distilling a small utility-based passage selector to enhance retrieval-augmented generation," in Proc. Annu. Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval Asia Pacific Region (SIGIR-AP), 2025.

[10] R. Sandhu, D. Mu, C. Chang, M. S. Tasjid, H. Rai, M. Volkovs, and G. Wu, "Aligning dense retrievers with LLM utility via distillation," arXiv preprint arXiv:2604.22722, 2026.

[11] H. Zhang, K. Bi, J. Guo, J. Zhang, S. Wang, D. Yin, and X. Cheng, "LLM-specific utility for retrieval-augmented generation," in Proc. 34th ACM Int. Conf. Inf. Knowl. Manage. (CIKM), 2026.

[12] L. Dai, Y. Xu, J. Ye, H. Liu, and H. Xiong, "SePer: Measure retrieval utility through the lens of semantic perplexity reduction," in Proc. Int. Conf. Learn. Representations (ICLR), 2025, spotlight.

[13] L. Perez-Beltrachini and M. Lapata, "Uncertainty quantification in retrieval augmented question answering," Trans. Mach. Learn. Res., 2025. [Online]. Available: https://openreview.net/forum?id=JLkgI0h7wy

[14] F. Tian, D. Ganguly, and C. Macdonald, "Predicting retrieval utility and answer quality in retrieval-augmented generation," arXiv preprint arXiv:2601.14546, 2026.

[15] M. Chandra, D. Ganguly, and I. Ounis, "LURE-RAG: Lightweight utility-driven reranking for efficient RAG," arXiv preprint arXiv:2601.19535, 2026.

[16] Y. Wu, X. Shen, F. Wang, C. Zhou, Z. Wu, X. Dai, and R. Xia, "Optimizing RAG rerankers with LLM feedback via reinforcement learning," arXiv preprint arXiv:2604.02091, 2026.

[17] H. Zhang, M. Tang, K. Bi, J. Guo, S. Liu, D. Shi, D. Yin, and X. Cheng, "Utility-focused LLM annotation for retrieval and retrieval-augmented generation," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2025.

[18] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov et al., "Dense passage retrieval for open-domain question answering," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2020, pp. 6769–6781.

[19] R. Friel, M. Belyi, and A. Sanyal, "RAGBench: Explainable benchmark for retrieval-augmented generation systems," arXiv preprint arXiv:2407.11005, 2024.

[20] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti et al., "Natural questions: A benchmark for question answering research," Trans. Assoc. Comput. Linguist., vol. 7, pp. 453–466, 2019.

[21] M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer, "TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension," in Proc. 55th Annu. Meeting Assoc. Comput. Linguist. (ACL), 2017, pp. 1601–1611.

[22] J. Berant, A. Chou, R. Frostig, and P. Liang, "Semantic parsing on Freebase from question-answer pairs," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2013, pp. 1533–1544. [Online]. Available: https://aclanthology.org/D13-1160

[23] C. Sciavolino, Z. Zhong, J. Lee, and D. Chen, "Simple entity-centric questions challenge dense retrievers," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2021, pp. 6138–6148.

[24] A. Mallen, A. Asai, V. Zhong, R. Das, D. Khashabi, and H. Hajishirzi, "When not to trust language models: Investigating effectiveness of parametric and non-parametric memories," in Proc. 61st Annu. Meeting Assoc. Comput. Linguist. (ACL), 2023, pp. 9802–9822.

[25] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2016, pp. 2383–2392.

[26] S. Min, J. Michael, H. Hajishirzi, and L. Zettlemoyer, "AmbigQA: Answering ambiguous open-domain questions," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2020, pp. 5783–5797.

[27] O. Press, M. Zhang, S. Min, L. Schmidt, N. Smith, and M. Lewis, "Measuring and narrowing the compositionality gap in language models," in Findings Assoc. Comput. Linguist.: EMNLP, 2023, pp. 5687–5711.

[28] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov et al., "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2018, pp. 2369–2380.

[29] X. Ho, A.-K. Duong Nguyen, S. Sugawara, and A. Aizawa, "Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps," in Proc. 28th Int. Conf. Comput. Linguist. (COLING), 2020, pp. 6609–6625.

[30] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "MuSiQue: Multihop questions via single-hop question composition," Trans. Assoc. Comput. Linguist., vol. 10, pp. 539–554, 2022.